DUOC UC

Escuela de Informática y Telecomunicaciones

INFORME DE EVIDENCIAS, MENSAJERÍA RABBITMQ Y CONFIGURACIÓN CLOUD

Evaluación Parcial N° 3 — Desarrollo Cloud Native I (DSY1107)

Asignatura: Desarrollo Cloud Native I (DSY1107)

Proyecto: Plataforma de Gestión Médica VidaSalud

Estudiantes: Dylan Rodríguez, Fernando Guerrero y Lucas Puentes

Docente:

Fecha: Octubre 2026

1. Introducción y Arquitectura General

En este informe presentamos el desarrollo y las pruebas de la Evaluación Parcial N° 3 de la asignatura Desarrollo Cloud Native I. Para esta entrega tomamos como base la plataforma médica **VidaSalud** que construimos en las etapas anteriores (con Angular 19, Microsoft Entra ID y AWS) y le agregamos mensajería asíncrona utilizando **RabbitMQ**, sin romper ni reemplazar la lógica original de agendamiento de citas médicas ni de catálogo.

Para cumplir con lo solicitado en el encargo, incorporamos dos microservicios nuevos al backend en Spring Boot: `ms-vidasalud-notifications` (encargado de escuchar las colas con `@RabbitListener`, confirmar mensajes con ACK manual y desviar errores a una Dead Letter Queue) y `ms-vidasalud-rabbit-admin` (que expone endpoints REST para crear y eliminar colas, exchanges y bindings validando los parámetros de entrada).

```
MICROSOFT AZURE (IDaaS)
- Tenant: VidaSalud (VidaSaludDUOC1.onmicrosoft.com)
- Tenant ID: d6edb470-bae6-41a0-b8c5-c2f55613b833
- App Backend: vidasalud-api (Client ID: 2590960d-413c-42a8-a1c7-fc1289f4fb44)
- App Frontend: vidasalud-frontend (Client ID: 3d3e2f6b-858d-4cf0-92ea-53497416e34d)
      |
      | (1) Emisión Access Token JWT
      v
CLIENTE / FRONTEND (Puerto 4200)
- Angular 19 + MSAL Angular
- Interceptor HTTP: Inyección de cabecera Authorization: Bearer <token>
      |
      | (2) Petición HTTP con Bearer Token
      v
AMAZON WEB SERVICES (AWS) / ENTORNO DOCKER
- AWS API Gateway (vidasalud-api-gateway) -> JWT Authorizer (Issuer y Audience)
- ms-vidasalud-bff (Puerto 8080) -> Validación de token JWT y Roles
- ms-vidasalud-catalog (Puerto 8082) -> Prestaciones médicas y cupos de boxes
- ms-vidasalud-appointments (Puerto 8081) -> Guarda citas en BD y publica eventos post-
commit
      |
      | (3) Publica eventos JSON (appointment.created / appointment.status.changed)
      v
BROKER RABBITMQ (Puerto 5672 AMQP / 15672 Management)
- Direct Exchange: vidasalud.appointments.exchange
    * Cola creación : vidasalud.notifications.queue (Routing Key: appointment.created)
    * Cola estados  : vidasalud.status.queue        (Routing Key:
appointment.status.changed)
- Dead Letter Exchange: vidasalud.appointments.dlx
    * Cola de fallos: vidasalud.notifications.dlq   (Routing Key: appointment.failed)
      |
      +--> ms-vidasalud-notifications (Puerto 8083): Consume con @RabbitListener y hace
ACK/NACK
      +--> ms-vidasalud-rabbit-admin (Puerto 8084): API REST para crear/borrar colas y
exchanges
```

1.1 Resumen de Componentes y Puertos

Componente	Puerto	Función en el proyecto
Angular 19 (Frontend)	4200	Interfaz web con login MSAL contra Microsoft Entra ID e interceptor de token JWT.
ms-vidasalud-bff	8080	Entrada principal del backend que valida el token JWT y los roles antes de redirigir peticiones.
ms-vidasalud-appointments	8081	Guarda las citas médicas en base de datos y actúa como productor publicando eventos en RabbitMQ después del commit.
ms-vidasalud-catalog	8082	Maneja las prestaciones médicas y la reserva de boxes cuando se confirma una cita.
RabbitMQ Broker	5672 / 15672	Recibe los mensajes en los exchanges y los enruta hacia las colas mediante bindings y routing keys.
ms-vidasalud-notifications	8083	Consumidor que escucha las colas con `@RabbitListener`, registra la notificación y ejecuta ACK o NACK manual.
ms-vidasalud-rabbit-admin	8084	Microservicio administrador con endpoints REST para crear y eliminar colas, exchanges y bindings.

2. Configuración e Identidad en Microsoft Entra ID y AWS

2.1 Creación del Inquilino (Tenant) VidaSalud

Para la autenticación de usuarios con IDaaS mantuvimos configurado nuestro inquilino del proyecto en Microsoft Entra ID:

- **Nombre de la organización:** VidaSalud
- **Dominio principal:** VidaSaludDUOC1.onmicrosoft.com
- **ID de inquilino (Tenant ID):** d6edb470-bae6-41a0-b8c5-c2f55613b833

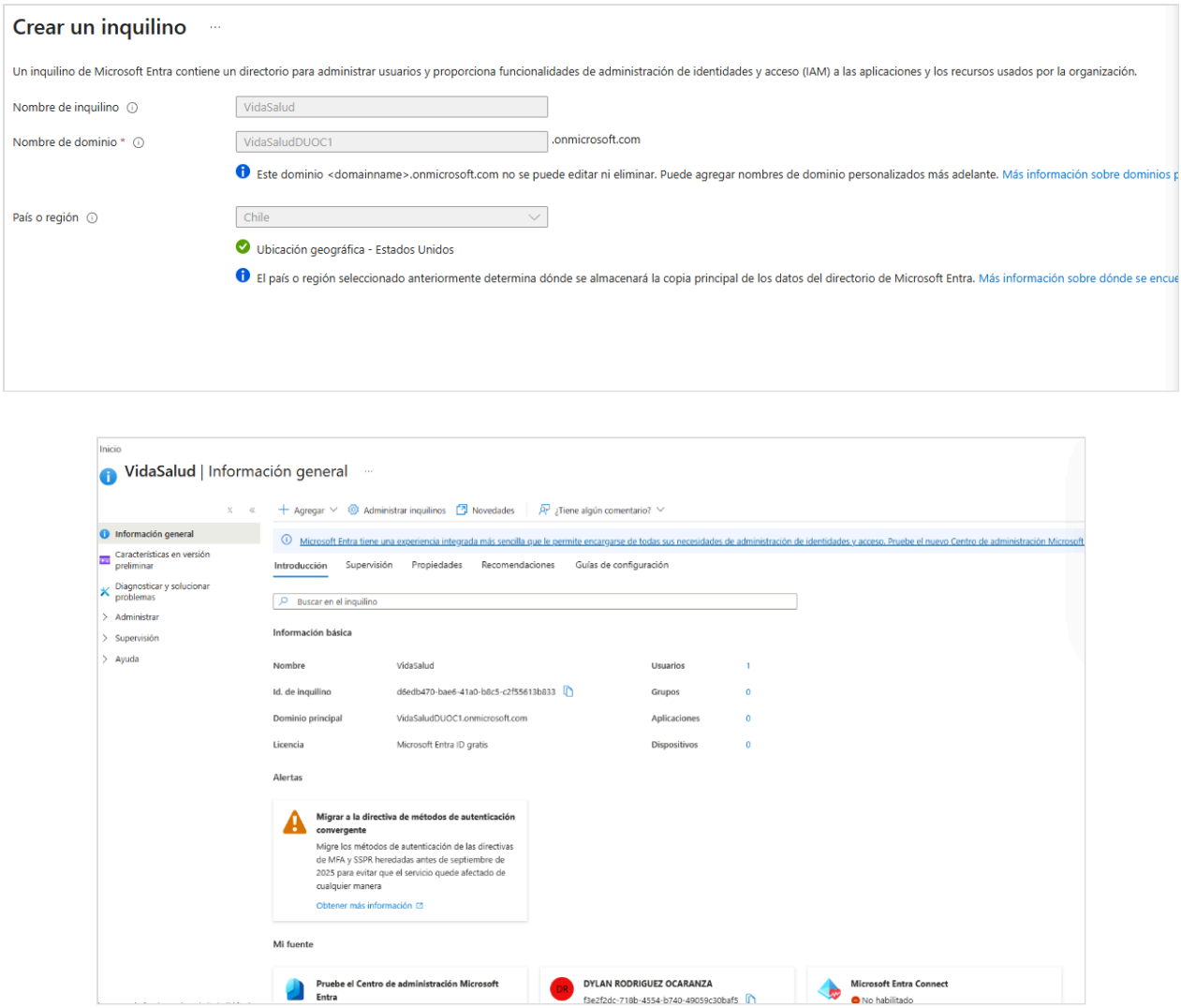


Figura 1. Información general del inquilino VidaSalud en Microsoft Entra ID mostrando el Tenant ID d6edb470-bae6-41a0-b8c5-c2f55613b833.

2.2 Registro de la API Backend (vidasalud-api) y Exposición de Scope

En los registros de aplicaciones configuramos el backend `vidasalud-api` y expusimos el ámbito delegado `access_as_user` para la emisión de tokens JWT:

- **Nombre de la aplicación:** vidasalud-api
- **ID de aplicación (Cliente):** 2590960d-413c-42a8-a1c7-fc1289f4fb44
- **URI del ID de aplicación:** api://2590960d-413c-42a8-a1c7-fc1289f4fb44
- **Ámbito expuesto:** access_as_user (Acceso a API VidaSalud)

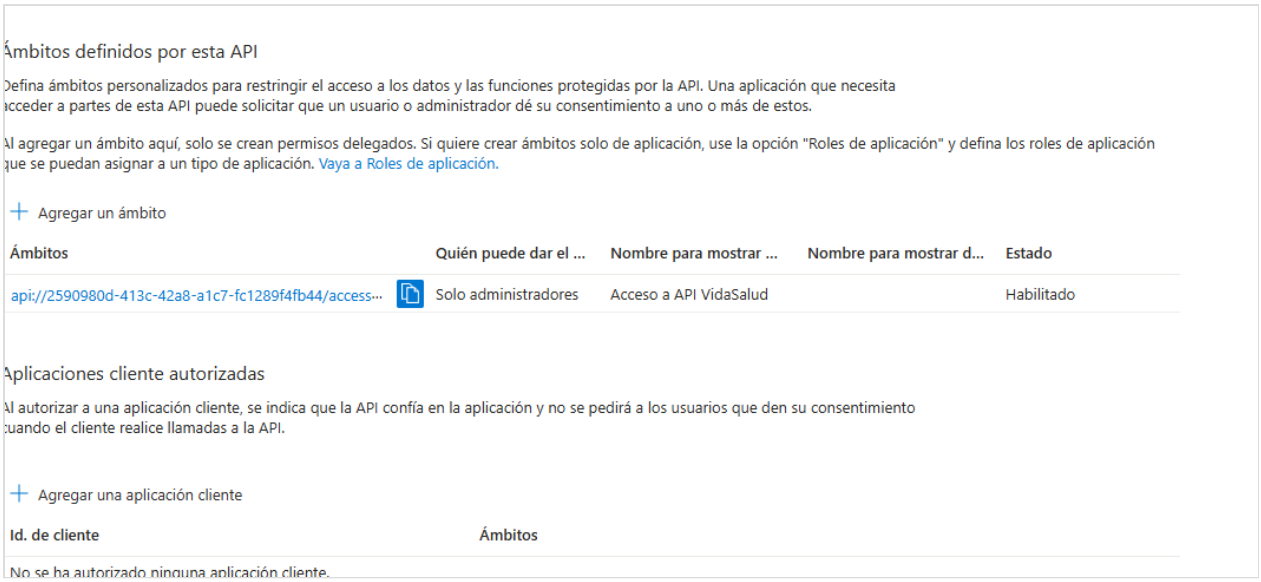


Figura 2. Configuración del ámbito access_as_user en Microsoft Entra ID.

2.3 Registro del Frontend (vidasalud-frontend) y Asignación de Permisos

También registramos la aplicación cliente `vidasalud-frontend` como una Single Page Application (SPA) apuntando a `http://localhost:4200` y le dimos el consentimiento de administrador para el permiso `access_as_user`:

- **Nombre de la aplicación:** vidasalud-frontend
- **ID de aplicación (Cliente):** 3d3e2f6b-858d-4cf0-92ea-53497416e34d
- **Plataforma:** SPA (Single Page Application) | Redirección: http://localhost:4200
- **Permiso asignado:** vidasalud-api / access_as_user

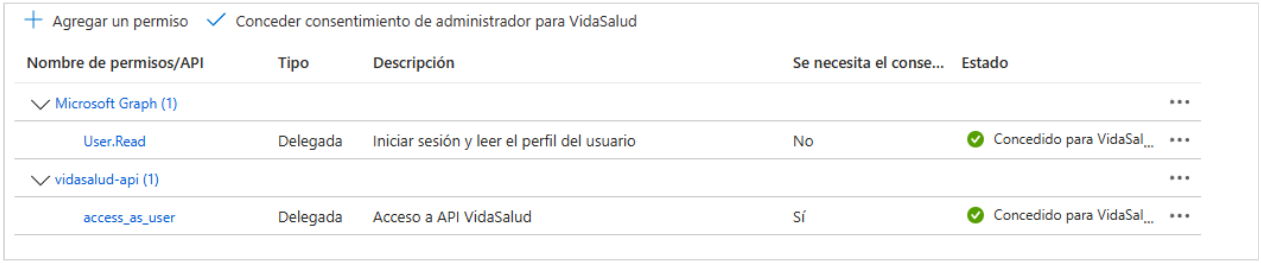


Figura 3. Permiso delegado api://2590960d-413c-42a8-a1c7-fc1289f4fb44/access_as_user concedido para VidaSalud.

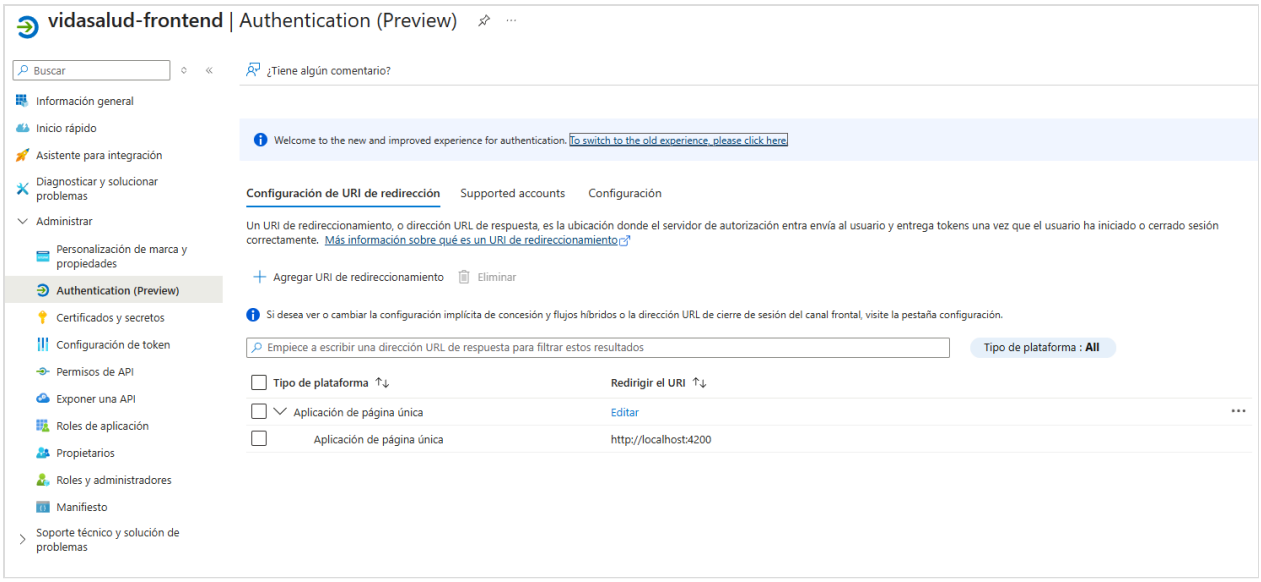


Figura 4. Configuración de URI de redirección SPA en http://localhost:4200 para vidasalud-frontend.

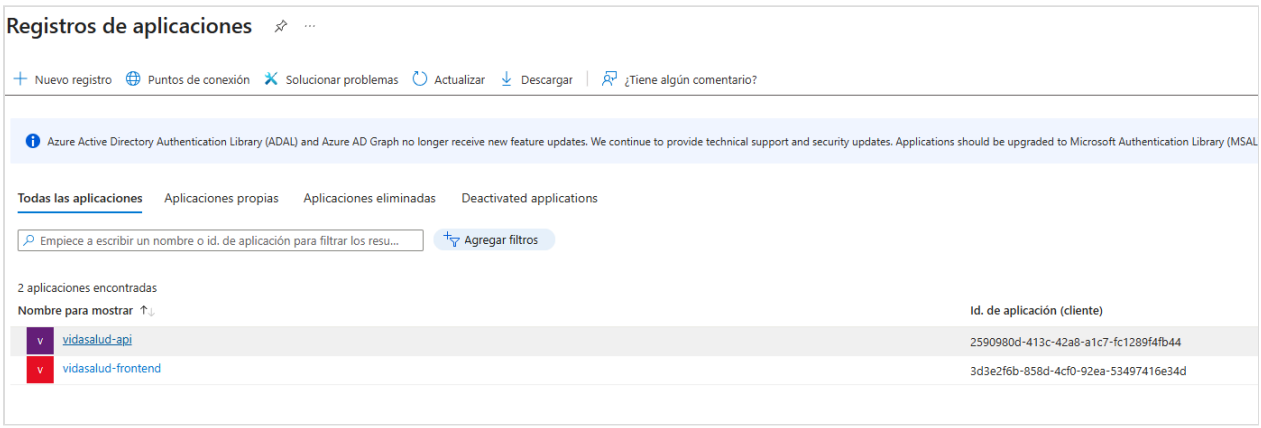


Figura 5. Aplicaciones vidasalud-api y vidasalud-frontend registradas en el inquilino.

2.4 Configuración de Perímetro en AWS API Gateway y EC2

En AWS configuramos la HTTP API `vidasalud-api-gateway` con el autorizador JWT `EntraID-JWT-Authorizer` que revisa el Issuer y el Audience antes de pasar las peticiones al contenedor del BFF en el puerto 8080. Los servicios internos de RabbitMQ (5672/15672), Notifications (8083) y Rabbit Admin (8084) quedan cerrados en loopback local (`127.0.0.1`) dentro del Docker Compose para que no queden expuestos a Internet.

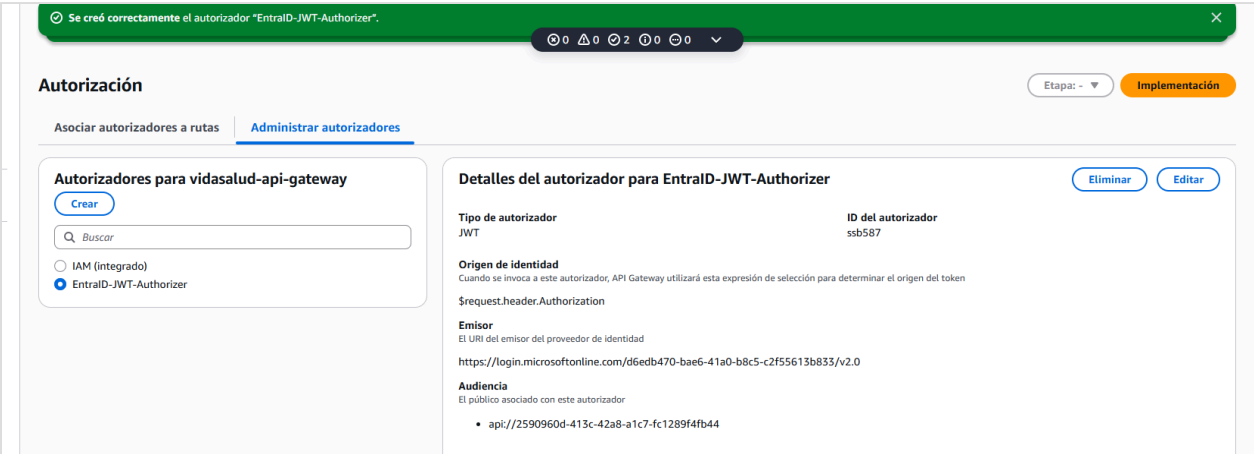


Figura 6. Configuración del autorizador JWT EntraID-JWT-Authorizer en AWS API Gateway.

3. Implementación de RabbitMQ según la Pauta de Evaluación

3.1 Nombres Centralizados en application.yml y RabbitNames

Para cumplir con el primer indicador de la pauta (no dejar textos ni nombres de colas sueltos en el código), definimos todos los nombres en `application.yml` y los cargamos con la clase `RabbitNames` usando `@ConfigurationProperties`:

```
# ms-vidasalud-notifications/src/main/resources/application.yml
vidasalud:
  rabbit:
    exchange: ${RABBITMQ_EXCHANGE:vidasalud.appointments.exchange}
    queue: ${RABBITMQ_QUEUE:vidasalud.notifications.queue}
    status-queue: ${RABBITMQ_STATUS_QUEUE:vidasalud.status.queue}
    dlx: ${RABBITMQ_DLX:vidasalud.appointments.dlx}
    dlq: ${RABBITMQ_DLQ:vidasalud.notifications.dlq}
    dead-routing-key: ${RABBITMQ_DEAD_KEY:appointment.failed}
    created-key: ${RABBITMQ_CREATED_KEY:appointment.created}
    status-key: ${RABBITMQ_STATUS_KEY:appointment.status.changed}
```

3.2 Beans de Queue, Exchange y Binding en RabbitConfig

En la clase `RabbitConfig.java` declaramos los beans de los dos exchanges directos, las tres colas durables y los tres bindings. En las colas de notificaciones y de estados agregamos los argumentos `x-dead-letter-exchange` y `x-dead-letter-routing-key` para que cuando un mensaje sea rechazado pase automáticamente a la DLQ:

```
@Configuration
@EnableConfigurationProperties(RabbitNames.class)
public class RabbitConfig {
    @Bean
    DirectExchange appointmentsExchange(RabbitNames names) {
        return new DirectExchange(names.exchange(), true, false);
    }

    @Bean
    DirectExchange deadLetterExchange(RabbitNames names) {
        return new DirectExchange(names.dlx(), true, false);
    }

    @Bean
    Queue notificationsQueue(RabbitNames names) {
        return QueueBuilder.durable(names.queue())
                .withArguments(Map.of("x-dead-letter-exchange", names.dlx(),
                        "x-dead-letter-routing-key", names.deadRoutingKey())).build();
    }

    @Bean
    Queue statusQueue(RabbitNames names) {
        return QueueBuilder.durable(names.statusQueue())
                .withArguments(Map.of("x-dead-letter-exchange", names.dlx(),
                        "x-dead-letter-routing-key", names.deadRoutingKey())).build();
    }

    @Bean
    Queue deadLetterQueue(RabbitNames names) {
        return QueueBuilder.durable(names.dlq()).build();
    }

    @Bean
    Binding createdBinding(RabbitNames names, DirectExchange appointmentsExchange, Queue
notificationsQueue) {
        return
BindingBuilder.bind(notificationsQueue).to(appointmentsExchange).with(names.createdKey());
    }

    @Bean
    Binding statusBinding(RabbitNames names, DirectExchange appointmentsExchange, Queue
statusQueue) {
        return
BindingBuilder.bind(statusQueue).to(appointmentsExchange).with(names.statusKey());
    }

    @Bean
    Binding deadBinding(RabbitNames names, DirectExchange deadLetterExchange, Queue
deadLetterQueue) {
        return
BindingBuilder.bind(deadLetterQueue).to(deadLetterExchange).with(names.deadRoutingKey());
    }
}
```

3.3 Separación de Negocio y Productor en Appointments

En el microservicio `ms-vidasalud-appointments` mantuvimos `AppointmentService` enfocado únicamente en guardar la cita y validar los cambios de estado, y separamos el envío de mensajes en la clase `AppointmentEventPublisher`. Además, usamos `afterCommit()` para asegurarnos de que el mensaje solo se envíe a RabbitMQ después de que la cita ya se guardó en la base de datos:

```
// AppointmentEventPublisher.java
public void publishAfterCommit(AppointmentResponse appointment, String routingKey) {
    Runnable send = () -> {
        try {
            Map<String, Object> event = Map.of(
                    "eventId", UUID.randomUUID().toString(),
                    "eventType", routingKey,
                    "occurredAt", Instant.now().toString(),
                    "appointment", appointment
            );
            rabbitTemplate.convertAndSend(exchange, routingKey, event, message -> {
                message.getMessageProperties().setMessageId((String) event.get("eventId"));
                message.getMessageProperties().setContentType("application/json");

message.getMessageProperties().setDeliveryMode(MessageDeliveryMode.PERSISTENT);
                return message;
            });
            log.info("Evento publicado {} cita {}", routingKey, appointment.id());
        } catch (Exception ex) {
            log.error("No fue posible publicar evento de cita {}", appointment.id(), ex);
        }
    };
    if (TransactionSynchronizationManager.isActualTransactionActive()) {
        TransactionSynchronizationManager.registerSynchronization(new
TransactionSynchronization() {
            @Override public void afterCommit() { send.run(); }
        });
    } else {
        send.run();
    }
}
```

3.4 Consumidores con @RabbitListener, ACK Manual y Envío a DLQ

En `ms-vidasalud-notifications` creamos `AppointmentConsumer` con `@RabbitListener` y modo de confirmación manual (`ackMode = "MANUAL"`). Cuando el mensaje es válido llamamos a `channel.basicAck(tag, false)`, y si ocurre una excepción (por ejemplo cuando probamos con `simulateFailure: true` o faltan datos), registramos el error en el log y llamamos a `channel.basicNack(tag, false, false)` con `requeue=false` para que vaya directo a la cola `vidasalud.notifications.dlq`:

```
@Component
public class AppointmentConsumer {
    @RabbitListener(queues = "${vidasalud.rabbit.queue}", ackMode = "MANUAL")
    public void onCreated(Message message, Channel channel) throws IOException {
consume(message, channel); }

    @RabbitListener(queues = "${vidasalud.rabbit.status-queue}", ackMode = "MANUAL")
    public void onStatus(Message message, Channel channel) throws IOException {
consume(message, channel); }

    private void consume(Message message, Channel channel) throws IOException {
        long tag = message.getMessageProperties().getDeliveryTag();
        try {
            JsonNode event = mapper.readTree(message.getBody());
            service.process(event);
            channel.basicAck(tag, false);
        } catch (Exception ex) {
            log.error("Mensaje rechazado y dirigido a DLQ: {}", ex.getMessage(), ex);
            channel.basicNack(tag, false, false);
        }
    }
}
```

3.5 Microservicio Administrador REST y Validación de Parámetros

En `ms-vidasalud-rabbit-admin` separamos los endpoints en `RabbitAdminController` y encapsulamos todas las operaciones de creación y borrado dentro de `RabbitAdminService` usando `AmqpAdmin`. Para evitar que se creen colas con nombres vacíos o tipos de exchange incorrectos, validamos los DTOs con `@Valid`, `@NotBlank` y `@Pattern`:

```
// QueueRequest.java
public record QueueRequest(
    @NotBlank @Pattern(regexp="[a-zA-Z0-9][a-zA-Z0-9._-]{0,127}") String name,
    boolean durable
) {}

// RabbitAdminController.java
@PostMapping("/queues")
@ResponseStatus(HttpStatus.CREATED)
public void createQueue(@Valid @RequestBody QueueRequest request) {
    service.createQueue(request);
}

@DeleteMapping("/queues/{name}")
@ResponseStatus(HttpStatus.NO_CONTENT)
public void deleteQueue(@PathVariable @Pattern(regexp="[a-zA-Z0-9][a-zA-Z0-9._-]{0,127}")
String name) {
    service.deleteQueue(name);
}
```

4. Pruebas Realizadas y Resultados Obtenidos

Levantamos el entorno con Docker Compose desde la carpeta `infra/apps` y ejecutamos las pruebas en PowerShell y en el panel web de RabbitMQ Management (`http://localhost:15672`) para comprobar cada punto pedido en la evaluación:

4.1 Comprobación de Contenedores y Creación de Cita Real

```
# 1. Estado de contenedores en Docker Compose
PS C:\ev3Cloud-main\infra\apps> docker compose -f compose.local.yml ps
NAME                    STATUS                    PORTS
apps-appointments-1     Up                        0.0.0.0:8081->8081/tcp
apps-catalog-1          Up                        0.0.0.0:8082->8082/tcp
apps-notifications-1    Up                        127.0.0.1:8083->8083/tcp
apps-rabbit-admin-1     Up                        127.0.0.1:8084->8084/tcp
apps-rabbitmq-1         Up (healthy)              127.0.0.1:5672->5672/tcp, 127.0.0.1:15672-
>15672/tcp

# 2. Creación de una cita mediante POST /api/appointments
PS C:\ev3Cloud-main\infra\apps> $body = @{
  patientId = "PAC-001"
  patientName = "Paciente Prueba"
  patientEmail = "prueba@vidasalud.cl"
  serviceId = 1
  scheduledAt = "2026-11-15T10:00:00-03:00"
} | ConvertTo-Json
PS C:\ev3Cloud-main\infra\apps> Invoke-RestMethod -Uri
"http://localhost:8081/api/appointments" -Method POST -ContentType "application/json" -Body
$body

# 3. Verificación en logs de Notifications y Appointments
appointments-1  | INFO AppointmentEventPublisher : Evento publicado appointment.created
cita 1
notifications-1 | INFO NotificationService : Notificación generada evento=test-001
tipo=appointment.created cita=1
```

4.2 Prueba de Mensaje Fallido hacia la DLQ y Validación del Administrador REST

```
# 1. Publicación de evento con fallo simulado (simulateFailure: true)
{"eventId": "test-error-001", "eventType": "appointment.created", "appointment": {"id":
1002}, "simulateFailure": true}
-> Log en Notifications: ERROR AppointmentConsumer : Mensaje rechazado y dirigido a DLQ:
Evento rechazado para prueba de DLQ
-> Resultado en RabbitMQ Management: 1 mensaje Ready en vidasalud.notifications.dlq

# 2. Prueba de creación de cola y validación de nombre vacío (HTTP 400) en Rabbit Admin
PS C:\ev3Cloud-main\infra\apps> $headers = @{ "X-Admin-Key" = "vidasalud-admin-local-2026-
key" }
PS C:\ev3Cloud-main\infra\apps> $body = @{ name = "vidasalud.test.queue"; durable = $true }
| ConvertTo-Json
PS C:\ev3Cloud-main\infra\apps> Invoke-RestMethod -Uri "http://localhost:8084/api/rabbit-
admin/queues" -Method POST -Headers $headers -ContentType "application/json" -Body $body

# Intentar crear cola con nombre vacío "":
PS C:\ev3Cloud-main\infra\apps> $bad = @{ name = ""; durable = $true } | ConvertTo-Json
PS C:\ev3Cloud-main\infra\apps> Invoke-RestMethod -Uri "http://localhost:8084/api/rabbit-
admin/queues" -Method POST -Headers $headers -ContentType "application/json" -Body $bad
-> Respuesta: HTTP 400 Bad Request ({"error":"Parámetros inválidos"})
```

4.3 Tabla Resumen de Pruebas Realizadas

Caso de Prueba	Acción / Endpoint	Resultado Esperado	Resultado Obtenido
Inicio de RabbitMQ y topología	`docker compose up -d` y acceso a puerto 15672	3 colas y 2 exchanges creados	Correcto: visibles en RabbitMQ Management.
Creación de cita y evento	`POST /api/appointments`	Cita guardada en H2 y evento recibido	Correcto: Notifications registró `appointment.created` y ejecutó ACK.
Cambio de estado de cita	`PUT /api/appointments/1/status`	Reserva de box y evento en cola de estado	Correcto: se publicó `appointment.status.changed` en `status.queue`.
Prueba de fallo y DLQ	Evento JSON con `simulateFailure: true`	Log de error, NACK y mensaje en DLQ	Correcto: mensaje enviado a `vidasalud.notifications.dlq`.
Crear y eliminar recursos por API	`POST` y `DELETE /api/rabbit-admin/queues`	HTTP 201 y HTTP 204	Correcto: la cola se creó y luego se eliminó del broker.
Validar cola con nombre vacío	`POST /api/rabbit-admin/queues` con `name: ""`	Rechazo con HTTP 400	Correcto: el backend devolvió HTTP 400 Bad Request.
Seguridad JWT en BFF / Gateway	Petición sin token / con rol PACIENTE a cambio de estado	HTTP 401 / HTTP 403	Correcto: bloqueado según las reglas de seguridad configuradas.

5. Repositorios y Código Fuente Entregado

El proyecto completo cuenta con sus archivos `.gitignore` en cada carpeta para no subir compilados ni dependencias pesadas:

1. **frontend-vidasalud:** Aplicación Angular 19 con MSAL Angular, Guards e Interceptor HTTP.
2. **ms-vidasalud-bff:** Backend for Frontend en Spring Boot con validación JWT de Microsoft Entra ID.
3. **ms-vidasalud-appointments:** Microservicio de citas médicas con JPA y publicación de eventos en RabbitMQ tras el commit.
4. **ms-vidasalud-catalog:** Microservicio de catálogo de prestaciones y boxes médicos.
5. **ms-vidasalud-notifications:** Microservicio consumidor de RabbitMQ con `@RabbitListener`, ACK/NACK manual y DLQ.
6. **ms-vidasalud-rabbit-admin:** Microservicio administrador REST con `RabbitAdminService` y validación de parámetros.
7. **infra:** Archivos Docker Compose para levantar todos los contenedores y RabbitMQ.

6. Conclusión

Con el desarrollo de esta tercera evaluación logramos integrar RabbitMQ dentro de la plataforma VidaSalud sin afectar el funcionamiento original de las citas médicas ni la seguridad configurada con Microsoft Entra ID y AWS. Pudimos comprobar en la práctica cómo desacoplar las notificaciones usando exchanges directos, colas durables y confirmaciones manuales (ACK/NACK), asegurando que cualquier mensaje con error pase a la cola Dead Letter (DLQ) sin bloquear el sistema, además de administrar dinámicamente las colas, exchanges y bindings mediante una API REST validada.